

Rockchip Developer Guide Linux GMAC DPDK

文件标识: RK-KF-YF-487

发布版本: V1.0.0

日期: 2023-03-06

文件密级: ☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自所有者所有。

版权所有 © 2023 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址: 福建省福州市铜盘路软件园A区18号

网址: www.rock-chips.com

客户服务电话: +86-4007-700-590

客户服务传真: +86-591-83951833

客户服务邮箱: fae@rock-chips.com

前言

概述

本文提供 Rockchip 平台以太网 GMAC 接口的 DPDK 介绍使用文档。

产品版本

芯片名称	内核版本
RK3568	4.19+

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	吴达超	2023-03-06	初始版本

目录

Rockchip Developer Guide Linux GMAC DPDK

1. 代码编译
 - 1.1 内核
 - 1.2 DPDK 编译
2. 运行 DPDK 程序
 - 2.1 挂载巨页
 - 2.2 加载 KO
 - 2.3 设置 performance 模式
 - 2.4 运行 testpmd
 - 2.5 运行 l2fwd
 - 2.6 运行 l3fwd
3. Pktgen
 - 3.1 下载 pktgen-dpdk 源码
 - 3.2 DPDK 编译
 - 3.3 Pktgen 编译
 - 3.4 运行 Pktgen 程序

1. 代码编译

1.1 内核

- 首先使能 DTS 中 UIO 节点，以 RK3568-evb1 参考：

```
diff --git a/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dtsi
b/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dtsi
index 0cb57e9d8529..c7729258e51d 100644
--- a/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dtsi
@@ -262,6 +262,14 @@
        status = "okay";
    };

+&gmac_uio0 {
+    status = "okay";
+};
+
+&gmac_uio1 {
+    status = "okay";
+};
+
/*
 * power-supply should switche to vcc3v3_lcd1_n
 * when mipi panel is connected to dsi1.
```

- 编译 KO

```
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 rockchip_linux_defconfig
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 menuconfig, 配置 CONFIG_UIO=m,
CONFIG_STMMAC_UIO=m, CONFIG_HUGETLBFS=y
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 rk3568-evb1-ddr4-v10-linux.img
-j8
烧写 boot.img
adb push drivers/uio/uio.ko, adb push
drivers/net/ethernet/stmicro/stmmac/stmmac_uio.ko
```

1.2 DPDK 编译

DPDK 测试使用的开发板跑的系统是 Debian 11，DPDK 版本 21.11，编译参考 http://doc.dpdk.org/guides-21.11/linux_gsg/cross_build_dpdk_for_arm64.html

- PC 交叉编译

```
wget https://developer.arm.com/-/media/Files/downloads/gnu-a/9.2-2019.12/binrel/gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu.tar.xz
tar -xvf gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu.tar.xz
export PATH=$PATH:<cross_install_dir>/gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu/bin
```

```
apt install build-essential
apt install pkg-config-aarch64-linux-gnu
apt-get install python3 python3-pip
apt install meson
apt install ninja-build
```

```
meson aarch64-build-gcc --cross-file config/arm/arm64_armv8_linux_gcc
meson --reconfigure aarch64-build-gcc --cross-file
config/arm/arm64_armv8_linux_gcc -Dbuildtype=debug -Dplatform=arm64 -
Dexamples=l2fwd,l3fwd
ninja -C aarch64-build-gcc
```

- 开发板编译 DPDK

```
apt-get install python3 python3-pip
apt install meson
apt install ninja-build
apt-get install libdpkg-perl
apt-get install build-essential
apt-get install python3-pyelftools

meson -Ddisable_drivers='common/cnxk' build
cd build
ninja
ninja install
```

2. 运行 DPDK 程序

2.1 挂载巨页

```
echo 1024 > /sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages
```

2.2 加载 KO

```
insmod uio.ko
insmod stmmac_uio.ko
```

默认开机起来后的网络走的还是原生内核网络，这里我们通过 `insmod stmmac_uio.ko` 和 `rmmod` 该 ko 来做到 DPDK 与内核原生网络的切换，即卸载 KO 后，会回到原来的内核网络状态。

- 进入 DPDK 网络控制：

```

root@linaro-alip:/home/linaro#insmod stmmac_uio.ko
[ 41.232761] Generic PHY stmmac-1:00: attached PHY driver [Generic PHY]
(mii_bus:phy_addr=stmmac-1:00, irq=POLL)
[ 41.236447] dwmac4: Master AXI performs any burst length
[ 41.236497] rk_gmac-dwmac fe010000.ethernet eth0: No Safety Features support
found
[ 41.236886] rockchip_eth_uio_drv fe010000.uio: Registered uio_eth0 uio
devices, 3 register maps attached
[ 41.241453] Generic PHY stmmac-0:00: attached PHY driver [Generic PHY]
(mii_bus:phy_addr=stmmac-0:00, irq=POLL)
[ 41.257084] dwmac4: Master AXI performs any burst length
[ 41.257138] rk_gmac-dwmac fe2a0000.ethernet eth1: No Safety Features support
found
[ 41.257523] rockchip_eth_uio_drv fe2a0000.uio: Registered uio_eth1 uio
devices, 3 register maps attached

```

- 返回内核网络控制:

```

root@linaro-alip:/home/linaro# rmmod stmmac_uio
[ 2200.304636] Generic PHY stmmac-0:00: attached PHY driver [Generic PHY]
(mii_bus:phy_addr=stmmac-0:00, irq=POLL)
[ 2200.318163] dwmac4: Master AXI performs any burst length
[ 2200.318205] rk_gmac-dwmac fe2a0000.ethernet eth1: No Safety Features support
found
[ 2200.318227] rk_gmac-dwmac fe2a0000.ethernet eth1: IEEE 1588-2008 Advanced
Timestamp supported
[ 2200.318443] rk_gmac-dwmac fe2a0000.ethernet eth1: registered PTP clock
[ 2200.319414] IPv6: ADDRCONF(NETDEV_UP): eth1: link is not ready
[ 2200.322971] Generic PHY stmmac-1:00: attached PHY driver [Generic PHY]
(mii_bus:phy_addr=stmmac-1:00, irq=POLL)
[ 2200.324883] dwmac4: Master AXI performs any burst length
[ 2200.324921] rk_gmac-dwmac fe010000.ethernet eth0: No Safety Features support
found
[ 2200.324945] rk_gmac-dwmac fe010000.ethernet eth0: IEEE 1588-2008 Advanced
Timestamp supported
[ 2200.325468] rk_gmac-dwmac fe010000.ethernet eth0: registered PTP clock
[ 2200.325814] IPv6: ADDRCONF(NETDEV_UP): eth0: link is not ready
[ 2205.385406] IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready

```

2.3 设置 performance 模式

```

echo performance > /sys/devices/system/cpu/cpufreq/policy0/scaling_governor
echo performance > /sys/devices/system/cpu/cpufreq/policy4/scaling_governor
echo performance > /sys/devices/system/cpu/cpufreq/policy6/scaling_governor
echo performance > /sys/class/devfreq/dmc/governor

```

2.4 运行 testpmd

- 转发模式测试命令
 - --vdev=net_stmmac0 --vdev=net_stmmac1 表示指定的虚拟设备，目前是名字是固定的
 - --main-lcore=0 表示 0 核用作管理，2 和 3 核用于转发

- --iova-mode=pa 因为设备不支持 iommu，故 iova-mode 规定为 pa 模式
- -- 用于分隔 eal 参数与 testpmd 的参数
- -i 表示进入 dpdk-testpmd 命令交互模式

```

root@linaro-alip:/home/linaro# ./dpdk-testpmd --iova-mode=pa --vdev=net_stmmac0
--vdev=net_stmmac1 -l 0,2,3 --main-lcore=0 -- -i
EAL: Detected CPU lcores: 4
EAL: Detected NUMA nodes: 1
EAL: Detected static linkage of DPDK
EAL: Multi-process socket /var/run/dpdk/rte/mp_socket
EAL: Selected IOVA mode 'PA'
TELEMETRY: No legacy callbacks, legacy socket not created
Interactive-mode selected
Warning: NUMA should be configured manually by using --port-numa-config and --
ring-numa-config parameters along with --numa.
testpmd: create a new mbuf pool <mb_pool_0>: n=163456, size=2176, socket=0
testpmd: preferred mempool ops selected: ring_mp_mc
Configuring Port 0 (socket 0)
stmmac_net: stmmac_eth_link_update()Port (0) link is Up
Port 0: BA:A0:3F:FD:B2:8C
Configuring Port 1 (socket 0)
stmmac_net: stmmac_eth_link_update()Port (1) link is Up
Port 1: B6:A0:3F:FD:B2:8C
Checking link statuses...
stmmac_net: stmmac_eth_link_update()Port (0) link is Up
stmmac_net: stmmac_eth_link_update()Port (1) link is Up
Done

```

- 开启转发模式测试

```

testpmd> start
io packet forwarding - ports=2 - cores=1 - streams=2 - NUMA support enabled, MP
allocation mode: native
Logical Core 2 (socket 0) forwards packets on 2 streams:
  RX P=0/Q=0 (socket 0) -> TX P=1/Q=0 (socket 0) peer=02:00:00:00:00:01
  RX P=1/Q=0 (socket 0) -> TX P=0/Q=0 (socket 0) peer=02:00:00:00:00:00

io packet forwarding packets/burst=32
nb forwarding cores=1 - nb forwarding ports=2
port 0: RX queue number: 1 Tx queue number: 1
  Rx offloads=0x0 Tx offloads=0x0
  RX queue: 0
    RX desc=0 - RX free threshold=0
    RX threshold registers: pthresh=0 hthresh=0 wthresh=0
    RX Offloads=0x0
  TX queue: 0
    TX desc=0 - TX free threshold=0
    TX threshold registers: pthresh=0 hthresh=0 wthresh=0
    TX offloads=0x0 - TX RS bit threshold=0
port 1: RX queue number: 1 Tx queue number: 1
  Rx offloads=0x0 Tx offloads=0x0
  RX queue: 0
    RX desc=0 - RX free threshold=0
    RX threshold registers: pthresh=0 hthresh=0 wthresh=0
    RX Offloads=0x0
  TX queue: 0

```

```
TX desc=0 - TX free threshold=0
TX threshold registers: pthresh=0 hthresh=0 wthresh=0
TX offloads=0x0 - TX RS bit threshold=0
```

- 查看转发数据:

```
testpmd> show port stats all

##### NIC statistics for port 0 #####
RX-packets: 5240160    RX-missed: 0    RX-bytes: 314409600
RX-errors: 0
RX-nombuf: 0
TX-packets: 0    TX-errors: 0    TX-bytes: 0

Throughput (since last show)
Rx-pps: 0    Rx-bps: 0
Tx-pps: 0    Tx-bps: 0
#####

##### NIC statistics for port 1 #####
RX-packets: 0    RX-missed: 0    RX-bytes: 0
RX-errors: 0
RX-nombuf: 0
TX-packets: 5180224    TX-errors: 0    TX-bytes: 310813440

Throughput (since last show)
Rx-pps: 0    Rx-bps: 0
Tx-pps: 0    Tx-bps: 0
#####

testpmd> show port stats all

##### NIC statistics for port 0 #####
RX-packets: 6382336    RX-missed: 0    RX-bytes: 382940160
RX-errors: 0
RX-nombuf: 0
TX-packets: 0    TX-errors: 0    TX-bytes: 0

Throughput (since last show)
Rx-pps: 681809    Rx-bps: 327268480
Tx-pps: 0    Tx-bps: 0
#####

##### NIC statistics for port 1 #####
RX-packets: 0    RX-missed: 0    RX-bytes: 0
RX-errors: 0
RX-nombuf: 0
TX-packets: 6322397    TX-errors: 0    TX-bytes: 379343820

Throughput (since last show)
Rx-pps: 0    Rx-bps: 0
Tx-pps: 681810    Tx-bps: 327269160
#####
```

- 其它常用设置:
 - --nb-cores 表示指定 dpdk-testpmd 用作转发工作的核的数量
 - --rxd 接收队列描述符

- --txd 发送队列描述符
- --rxq 表示指定 dpdk-testpmd 接收队列数，RK3568 队列为1
- --txq 表示指定 dpdk-testpmd 发送队列数，RK3568 队列为1

2.5 运行 l2fwd

l2fwd 默认至少要有两个核才能测试转发

```
./dpdk-l2fwd -l 0,2,3 --main-lcore=0 --iova-mode=pa --vdev=net_stmmac0 --
vdev=net_stmmac1 -- -q 1 -p 0x3
```

l2fwd 运行的串口信息每隔10s钟会刷新一次，考虑可能会导致丢包，建议将串口信息重定向到文件。

2.6 运行 l3fwd

```
./dpdk-l3fwd -l 3 -n 1 --iova-mode=pa --vdev=net_stmmac0 --vdev=net_stmmac1 -- -
p 0x3 -P --config="(0,0,3),(1,0,3)" --parse-ptype
```

- -p PortMask 参数指定使用的网口掩码
- -P 参数表示将所有网口设置为混杂模式，以便收到所有数据包
- --config (port,queue,lcore), [(port,queue,lcore)] 参数用以配置网口、队列、核之间的对应关系，例如，--config (0,0,3) 表示网口 0 的队列 0 由核 3 进行处理

值得注意的是，上述输出中打印了 l3fwd 的默认路由规则，即

```
LPM: Adding route 198.18.0.0 / 24 (0) [net_stmmac0]
LPM: Adding route 198.18.1.0 / 24 (1) [net_stmmac1]
LPM: Adding route 2001:200:: / 64 (0) [net_stmmac0]
LPM: Adding route 2001:200:0:1:: / 64 (1) [net_stmmac1]
```

也就是说，目的 IP 为 198.18.0.0/24 段的数据包将会通过网口 0 进行转发，目的 IP 为 198.18.1.0 / 24 段的数据包将会通过网口 1 进行转发。上述默认路由规则是在源码中配置的，所以在 l3fwd 测试的时候，需要设置好测试数据的目标 ip 和源 ip。

3. Pktgen

在板子上跑 Pktgen，是基于 dpdk，所以需要先编译和安装好 DPDK，编译参考 1.2 章节的开发板编译 DPDK。

3.1 下载 pktgen-dpdk 源码

```
git clone http://dpdk.org/git/apps/pktgen-dpdk
apt-get install libpcap-dev
apt-get install libnuma-dev
apt install meson
apt install ninja-build pkg-config
```

3.2 DPDK 编译

```
cd build
ninja
ninja install
ldconfig
export PKG_CONFIG_PATH=/usr/local/lib/aarch64-linux-gnu/pkgconfig/
```

3.3 Pktgen 编译

```
cd pktgen-dpdk
meson build
cd build
ninja
```

3.4 运行 Pktgen 程序

```
./build/app/pktgen --iova-mode=pa --vdev=net_stmmac0 -l 6,7 --proc-type auto --log-level debug -- -P -m 7.0
```

其中，EAL options 参数部分可以参看 DPDK EAL parameters，最重要的一个参数就是 -l 参数，用它来指定使用的核列表，比如：-l 1,2 或者 -l 1-2，表示使用核 1 和核 2。

值得注意的是，pktgen 至少要指定两个核，因为 pktgen 需要一个核与用户进行交互，比如响应测试过程中用户的输入。

pktgen 自有参数部分最重要的是 -m 参数，用它来指定网口与核之间的对应关系，比如：

-m 2.0：表示让核 2 来处理网口 0。值得注意的是，若要指定多个对应关系（使用多个网卡和多个核），则需多次使用 -m 参数。

如果要收包，最好也指定一下 -P 参数，表示让所有网口进入混杂模式，以便接收到所有数据包。

设置数据包格式并开启 Pktgen：

```
set 0 size 64
set 0 src ip 198.18.0.100/24
set 0 dst ip 198.18.1.101
set 0 dst mac ba:a0:3f:fd:b2:8c
start 0
```